

Code-Layout, Readability and Reusability

Date	29 September 2026
Team ID	SWTID-2026-7019
Project Name	EduGenie - AI Learning Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	4-space indentation in every Python module; consistent 2-space indentation in HTML/JS/CSS.
2	Proper File Structure	Files and folders are logically organized	Partial	One module per feature at project root, plus static/ and templates/. No tests/ or docs/ folder yet, and compiled __pycache__ / files are tracked in git.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. GEMINI_API_KEY, quiz_text, cleaned_text, explain_tokenizer, window._quizScore.
4	Function / Method Names	Functions are descriptively named	Yes	e.g. answer_question_with_gemini(), generate_quiz(), summarize_text(), get_learning_recommendations(), explain_topic().
5	Code Comments	Inline and block comments explain logic	Partial	Section banners and route comments in main.py and index.html; feature modules have few comments and no docstrings.
6	Modular Design	Code is split into reusable functions/modules	Yes	Each AI feature lives in its own module and is imported by main.py; JS helpers setLoading(), showResult(), showToast().
7	No Redundant Code	Duplicate or unused code is removed	Partial	sys.path line and Gemini client set-up are repeated in every module; default model names differ between modules.
8	Error Handling	Exceptions and errors are handled gracefully	Partial	try/except in every Gemini module, API-key guard, 400 validation and a global JSON handler; module errors are returned as text (HTTP 200) and the local-model path has no try/except.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	clean_json_block()	Python (re)	quiz_module.py; reusable for any LLM output wrapped in ```json fences	Medium
2	_check_api_key() guard	Python / FastAPI	Called by all five API routes in main.py	High
3	Gemini call modules: qna, summary_module, quiz_module, learning_path	Python (google-genai)	Same (input, api_key, model) signature; can be plugged into any new route or app	High
4	explain_topic() with local-model to Gemini fallback	Python (Transformers, PyTorch)	Explanation route; pattern reusable for other local-with-cloud-fallback features	Medium
5	setLoading(), showResult(), showToast()	JavaScript	All five tool cards in templates/index.html	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear one-module-per-feature layout; no tests folder and tracked build artefacts.
Readability	4	Descriptive names, consistent indentation, short functions.
Reusability	3	Shared call pattern and JS helpers, but client set-up and sys.path code are duplicated.
Documentation / Comments	2	Section comments only; no docstrings and no README.
Overall Score	3.3 / 5	Average of the four ratings; documentation and de-duplication are the main improvements.